



Transaction Processing

Talia Noreen

**Beaconhouse National University
School of Computer and Information Technology (SCIT)**

Transaction Processing

- A Transaction is a logical unit of work on the database.
- A Transaction may be an entire program, a portion of a program or even a single statement.
- A Transaction brings the database from one consistent (correct/acceptable) state to another consistent state.
- Consistent state is one in which all data integrity constraints are satisfied.
- All transactions are controlled and executed by the DBMS to guarantee database integrity.
- While a transaction is in progress, the database may temporarily be in some inconsistent state.

Transaction Processing

- Consider the example of a banking transaction involving two relations:
 1. ACC_TRANS(Acc_No, Amount, Date, Deposit/Withdraw, ...)
 2. ACC_BAL(Acc-No, Balance,...)
- Against every insertion in ACC_TRANS, the ACC_BAL must be updated appropriately, otherwise the database will be in some inconsistent or wrong state.

Example Transaction

- Register credit sale of 100 units of product X to customer Y for \$500
 - Reduce product X's quantity by 100
 - Adding \$500 to customer Y's account receivable

```
UPDATE PRODUCT
SET PROD_QTY = PROD_QTY - 100
WHERE PROD_CODE = 'X';
UPDATE ACCT_RECEIVABLE
SET ACCT_BALANCE = ACCT_BALANCE + 500
WHERE ACCT_NUM = 'Y';
```

Transaction Processing

- From the above examples, it is observed that, there can be one of two possible outcomes of a transaction:
 1. **Committed:** A Successful completion of a transaction will bring the database into a new consistent state.
 2. **Aborted:** An incomplete transaction will leave the database in some inconsistent state.
- From 1 & 2, it can be concluded that a transaction must be **entirely completed** or **entirely aborted**.
- An aborted transaction must be **undone or rolled backed** to keep the database in some consistent state.
- A System component called **Transaction Manager** or **Recovery Manager** supports the above feature.

Properties of Transactions

A TOMICITY

⇒ all or nothing

CONSISTENCY

⇒ no violation of integrity constraints

ISOLATION

⇒ concurrent changes invisible È serializable

DURABILITY

⇒ committed updates persist

Transaction Properties

—Atomicity

- Either **all or none** of the transaction's operations are performed.
- Atomicity requires that if a transaction is interrupted by a failure, its partial results must be **undone**.

Consistency

- Internal consistency
 - ⇒ A transaction which executes *alone* against a *consistent* database leaves it in a consistent state.
 - ⇒ Transactions do not violate database integrity constraints.
- Transactions are **correct** programs

Transaction Properties

Isolation

■ Serializability

- ➡ If several transactions are executed concurrently, the results must be the same as if they were executed serially in some order.

■ Incomplete results

- ➡ An incomplete transaction cannot reveal its results to other transactions before its commitment.
- ➡ Necessary to avoid cascading aborts.

Transaction Properties

Durability

- Once a transaction commits, the system must guarantee that the results of its operations will never be lost, in spite of subsequent failures.
- Database recovery

COMMIT, ROLLBACK and SAVEPOINT

- Statements for transaction management.
- A transaction begins with the first DML statement or by using an explicit **SET TRANSACTION** statement and ends when a **COMMIT** or **ROLLBACK** statement is issued.
- Use COMMIT and ROLLBACK SQL statements to terminate a transaction explicitly.
- The syntax for COMMIT statement is:
 - **COMMIT [WORK][COMMENT text];**
- The optional word WORK is used just to improve readability.
- The COMMENT is usually employed by distributed transactions.
 - **COMMIT COMMENT 'Employees details have been changed';**

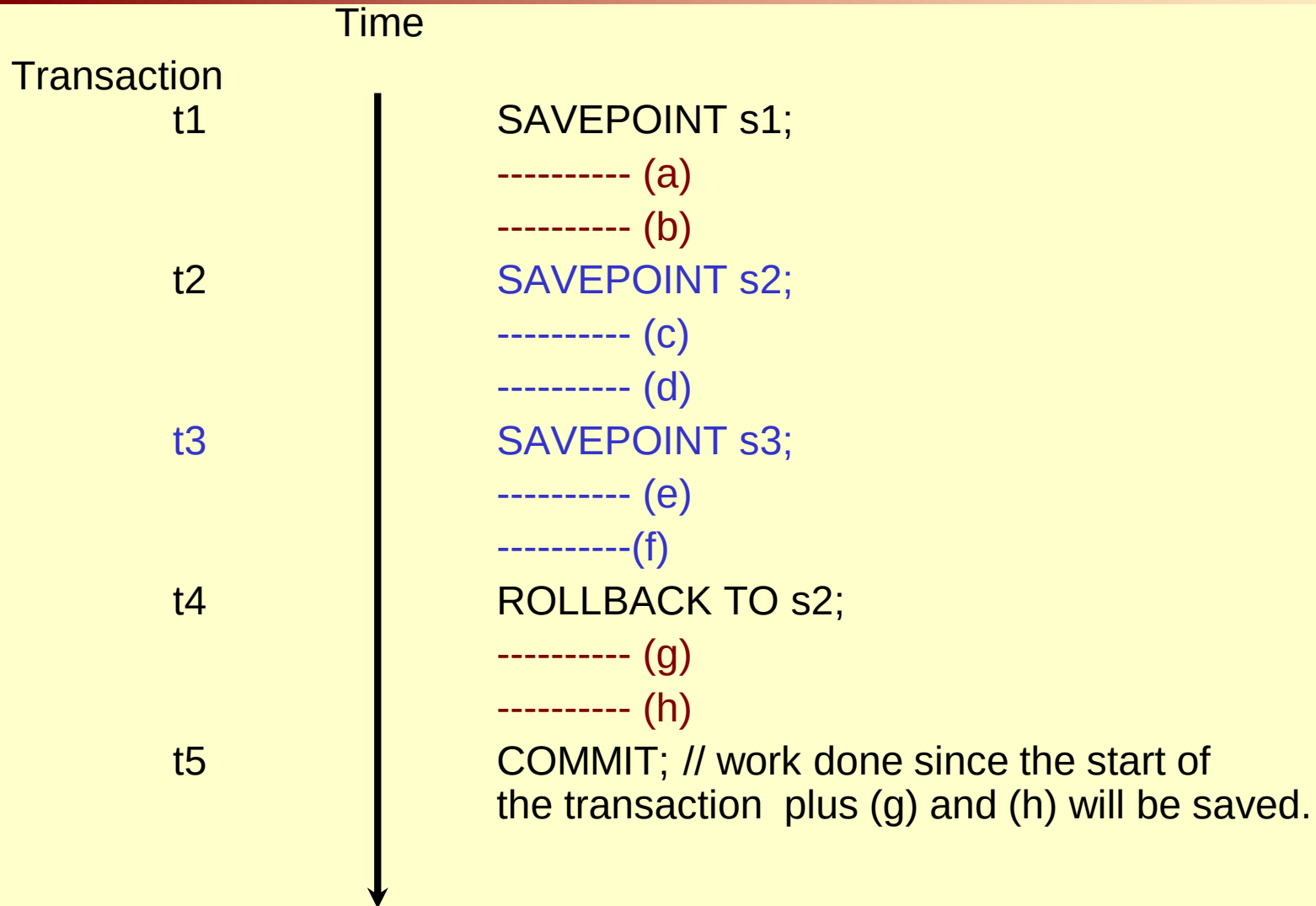
COMMIT, ROLLBACK and SAVEPOINT

- SVAEPOINT gives more control on transaction management.
- SVAEPOINT marks and saves the current point in a transaction.
- This allows you to rollback the changes up to that point.
- There can be maximum 5 savepoints in a transaction.
- The syntax for SAVEPOINT statement is:
 - SVAEPOINT savepoint-name;
 - SVAEPOINT Emp_savepoint;

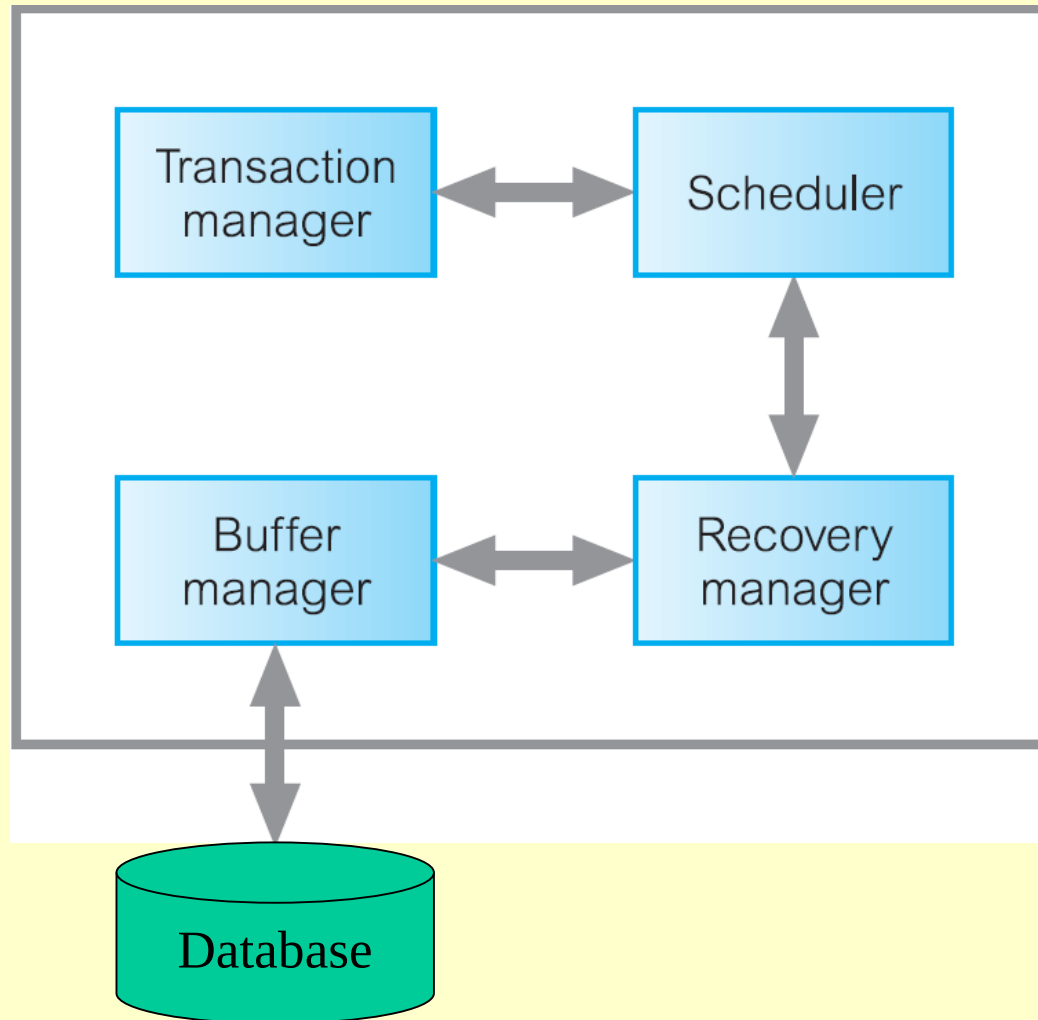
COMMIT, ROLLBACK and SAVEPOINT

- ROLLBACK statement is used to undo the changes made by the transaction. However, it can't undo the changes that have been made permanent by COMMIT statement.
- The syntax for ROLLBACK statement is:
 - `ROLLBACK [WORK][TO savepoint_name];`
- Examples:
- `ROLLBACK;` // it will undo the changes made by the current transaction (up to the last COMMIT)
- `ROLLBACK TO rollback_point;`
- // it will undo the changes up to the specific savepoint.

COMMIT, ROLLBACK and SAVEPOINT (Example)



Transaction Management



Buffer Management

- An DML operation can be regarded as permanent only after the contents of **memory buffer** are flushed to secondary storage.
- The flushing of the buffers to the database can be triggered by a specific command **(COMMIT)** or automatically when the buffers become full. The explicit writing of the buffers to secondary storage is known as **force-writing**.
- For efficient buffer management, the following smart measures are used:
 - **Present / absent bit**
 - **Dirty bit**
 - **Page replacement algorithms**

Database Recovery

- Database recovery is a process of restoring a database from a given state, usually inconsistent, to some consistent (correct) state in the event of a failure.
- The **recovery manager** of a DBMS is responsible for ensuring two important properties of a transaction: **Atomicity and Durability**.
- It ensures atomicity by undoing the actions of transactions that do not commit and durability by making sure that all actions of committed transactions survive system crashes.
- It ensures **atomicity** by undoing the actions of transactions that do not commit (**Roll Backward or Undo**)
- It ensure **durability** by making sure that all actions of committed transactions survive even if the system crashes (**Roll Forward or Redo**).

Database Recovery Facilities

1. Backup mechanism
2. Logging facility
3. Checkpoint facility

Database Recovery Facilities

1. Backup mechanism:

- An Automatic dump facility that produces a backup copy of the entire database.
- The backup copy is used to restore the database in the event of data loss or damage.

a) Full Backup

b) Differential Backup: Only the last modifications to the database (when compared with a previous backup copy).

Database Recovery Facilities

2. Logging facility:

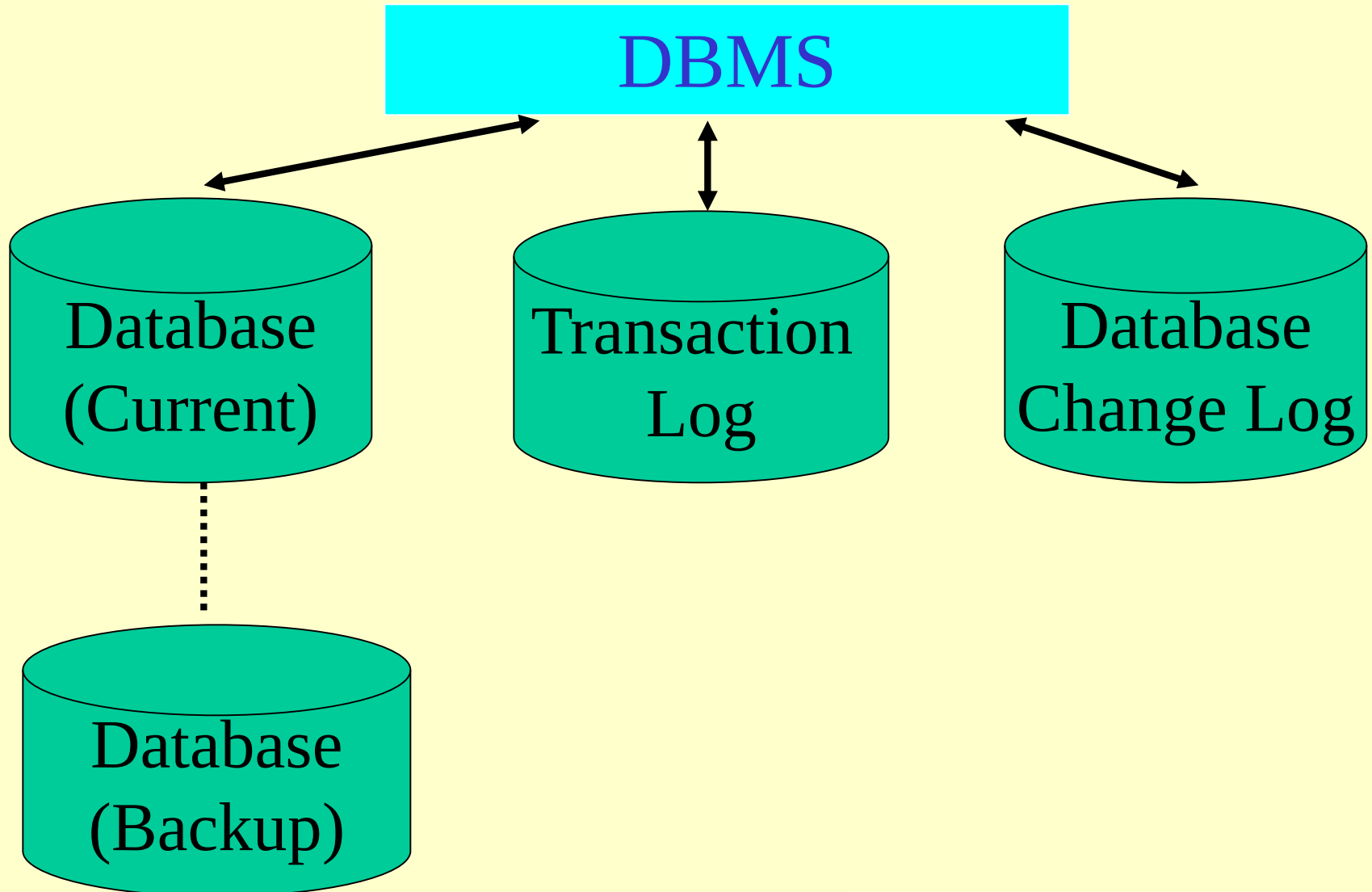
- It is also called **trail** or **journal**.
- It maintains history of actions executed by the DBMS.
- Owing to the importance of log, two or more copies of the log on different disks or different locations of the same disk are maintained.
- The most recent portion of the log, called **log tail** is kept in main memory and periodically flushed to stable storage.
- Every log record is assigned a unique id called **Log Sequence Number (LSN)**.
- A log record is written for each command such as insert, update, delete, undo, commit, savepoint etc.

Database Recovery Facilities

2. Logging facility:

- Two types of logs are used for database recovery.
 - a) Transaction Log
 - b) Database Change Log
- Transaction Log stores the information like Transaction Code, Time, Terminal, User, input data values, records accessed and modified.
- Database Change Log contains **before images** and **after images** of records that have been modified by transactions.
- Log Records are written before the transaction is actually committed and it contains both old and updated values.
- This is known as **Write Ahead Log Protocol**.

Database Recovery Facilities



Database Recovery Methods

- The particular recovery method to be used depends on the extent of the damage that has occurred to the database.
- If the database has been extensively damaged, for example a disk head crash has occurred and destroyed the database, then **Restore/Rerun** technique is used.
- If the database has not been physically damaged but has become inconsistent, then either **forward recovery** or **backward recovery** is used.

Database Recovery Methods

1. Restore:

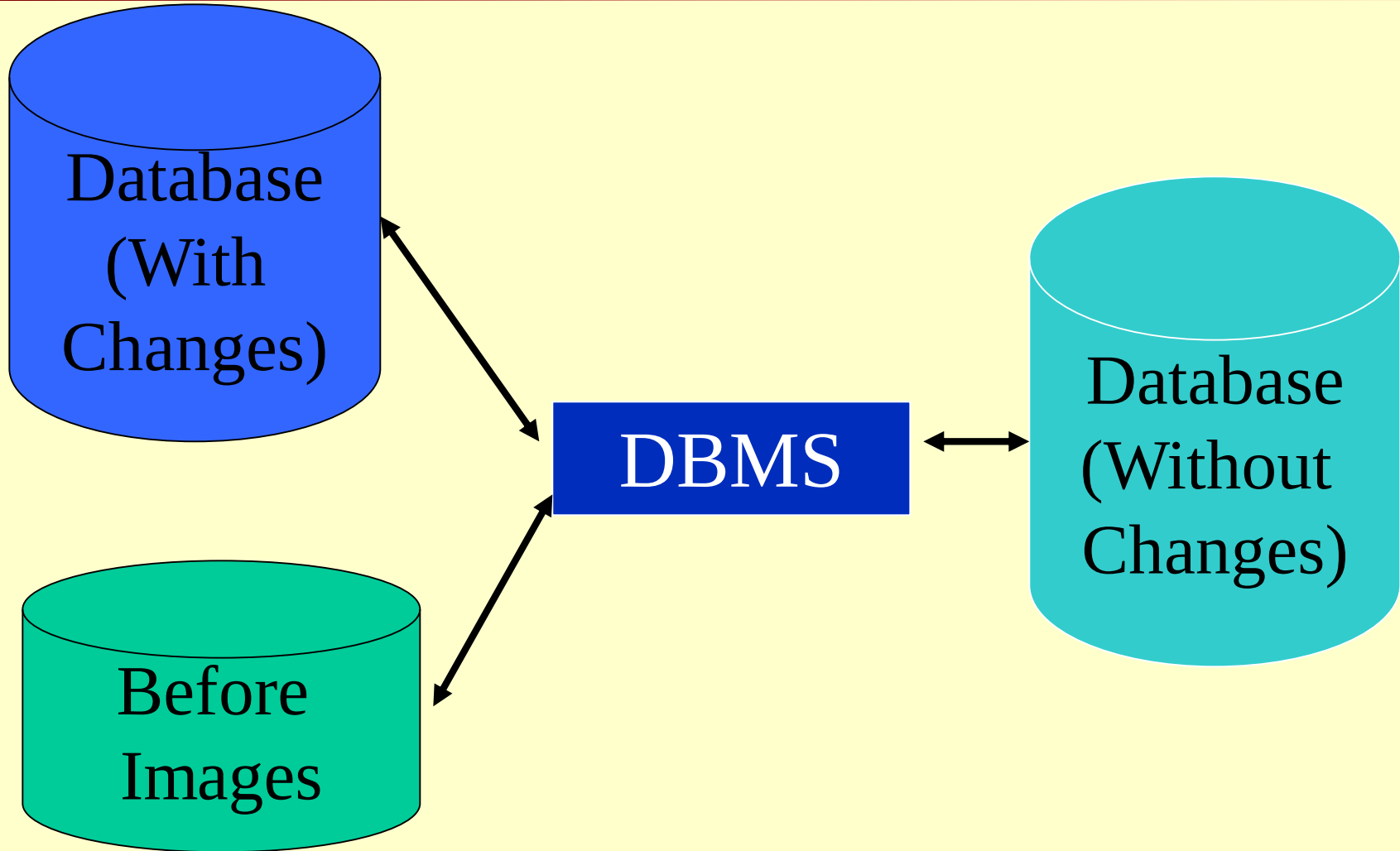
- The Restore/Rerun technique involves reprocessing the transaction up to the point of failure against the backup copy of the database.
- The most recent backup copy of the database is mounted and all transactions(stored on transaction log) that have occurred since the copy are rerun.

Database Recovery Methods

2. Backward Recovery (Roll Backward)

- It is also called immediate update recovery protocol.
- Updates are applied to the database as they occur without waiting to reach a COMMIT point.
- If a transaction fails before it reaches this point, we need to undo the changes.
- It is used to undo unwanted changes to the database.
- Before images of the records that have been changed are applied to the database.
- As a result, the database is returned to an earlier state and the unwanted changes are eliminated.
- This is shown in the Fig:

Roll Backward

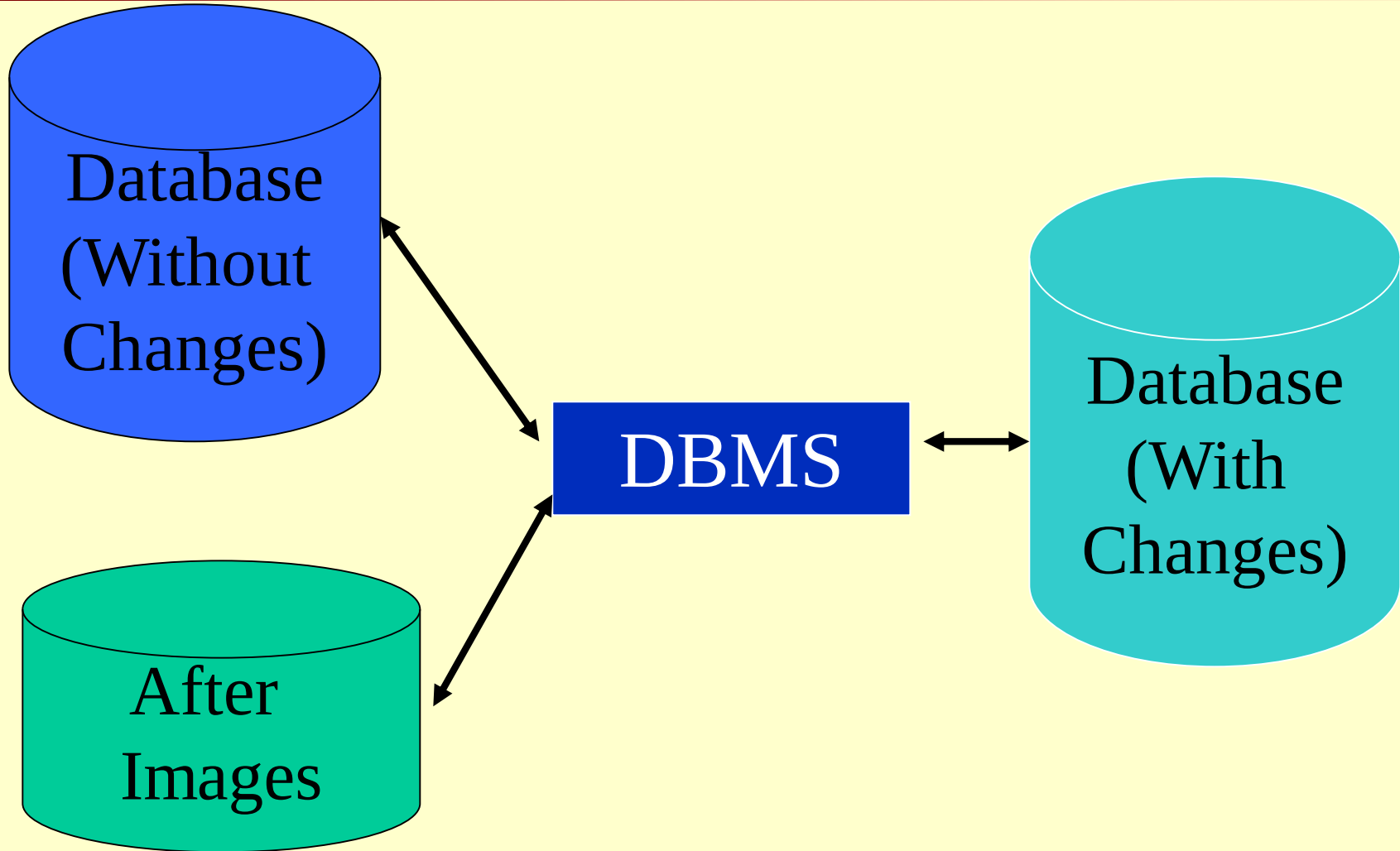


Database Recovery Methods

3. Forward Recovery (Roll Forward)

- It is also called deferred update recovery protocol.
- Updates are deferred and not written to the database immediately until after a transaction has reached its COMMIT point.
- If a transaction fails before it reaches this point, it would not have modified the database and so no undoing of changes will be necessary. However, it may be necessary to redo the updates of committed transactions.
- It is used to redo the changes that could not be stored to the database because of some failure during a transaction.
- After images of the records that could not be changed are applied to the database.
- As a result, the database is forwarded to a later state.
- This is shown in the Fig:

Roll Forward



Database Recovery

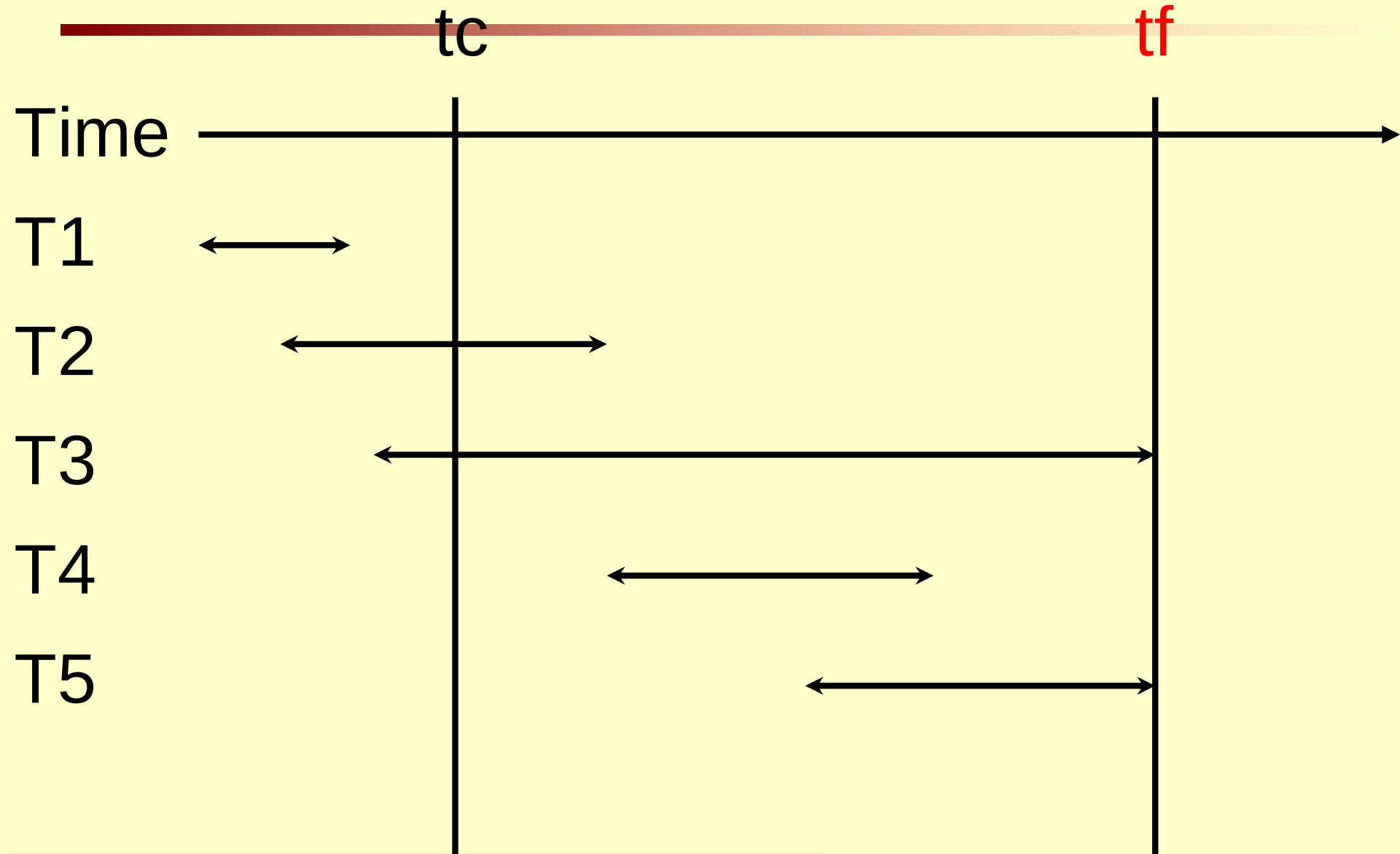


- When to use Roll Backward and When to use Roll Forward?
- Which Transaction to undo and which to redo?

Checkpointing

- A difficulty in recovering from log file is that when a failure occurs we may not know how far back in the log to search.
- The DBMS records **checkpoints** (at certain prescribed intervals) to the log file whenever it writes all of its updated buffers to the disk. Checkpoints are used to synchronize the log file.
- A DBMS may perform checkpoints automatically or in response to commands in user application programs.
- When failures do occur, the recovery is resumed from the most recent checkpoint.

Undo & Redo



Undo & Redo

- All transactions T, for which both <T Starts> and <T Commit> appear in the log are **redone**.
- For transactions S, for which the log contains <S Starts> but not <S Commit> will be **undone**.
- In the Fig above, transactions T3 and T5 must be **undone** and transactions T2 and T4 must be **redone**.
- At restart, the system goes through the following procedure:

Undo & Redo

- It starts with two lists of transactions, the UNDO list and REDO list.
- UNDO list is set to all transactions recorded at the time of checkpoint, whereas REDO list is set to empty.
- Log is searched starting from the checkpoint record.
- If <T Starts> for a transaction T is found in the log, add it to UNDO list.
- If <T Commit> for a transaction T is found, move T from UNDO list to REDO list.
- When the end of log is reached, the UNDO and REDO lists are identified.

Reasons of Database Failure

1. System Failure
2. Database Destruction (Media Failure)
3. Application software errors
4. Incorrect data
5. Natural physical disasters
6. Sabotage

Reasons of Database Failure

1. System Failure

- It affects all transactions currently in progress but does not physically damage the database.
- It is also known as soft crash.
- Some causes of system failure are power loss, system software failure etc.
- System failure can be recovered using Roll Forward and Roll Backward.

2. Database Destruction:

- Database itself is lost or destroyed.
- It is also known as Hard Crash.
- A Backup copy of the database is required for recovery in this situation.

Reasons of Database Failure

3. Application software errors

- Logical errors in a program that cause one or more transactions to fail.

4. Incorrect Data

- Database updated with an incorrect data may initiate a chain to erroneous transactions leading to database failure.
- In such situation, the database can be recovered using:
 - a) Backward Recovery
 - b) Applying a series of compensating transactions to nullify the effect of incorrect input.



Thank you for your attention.

Talia Noreen

Lecturer

Beaconhouse National University

School of Computer and Information Technology (SCIT)

13 Km Niazbeg Raiwind Road (off Thokar, Beacon House Society

Lahore, Pakistan

E-mail: talia.noreen@bnu.edu.pk